



Memory

Short-Term Memory

Overview

For this LangGraph uses checkpointing...

- These are snapshots of the state
- They are saved at each superstep
- Checkpoints are persisted and can be used to restore the state of a thread at a later time
- **A thread saves all the snapshots in a graph run to the same ID**
 - When invoking a graph with a checkpointer, you **must** specify a `thread_id` as part of the `configurable` portion of the config.
 - Note: if using subgraphs, you only need to provide the checkpointer when compiling the parent graph. LangGraph will automatically propagate the checkpointer to the child subgraphs.

▼ InMemory Checkpointer Demo

```
checkpointer = InMemorySaver()
graph = workflow.compile(checkpointer=checkpointer)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}
graph.invoke({"state_field": ""}, config)
```

▼ PostgreSQL Checkpointer Demo

- In production we would use a database checkpointer
- I.e. using PostgreSQL here or we can use MongoDB, Redis etc

```
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
```

```
builder = StateGraph(...)  
graph = builder.compile(checkpointer=checkpointer)
```

Using Checkpoints

▼ View a / All Snapshot(s) Demo

```
# get the latest state snapshot
config = {"configurable": {"thread_id": "1"}}
graph.get_state(config)

# get a state snapshot for a specific checkpoint_id
config = {"configurable": {"thread_id": "1", "checkpoint_id": "1ef6
63ba-28fe-6528-8002-5a559208592c"}}
graph.get_state(config)

# get full state history
config = {"configurable": {"thread_id": "1"}}
list(graph.get_state_history(config))
```

▼ Time-Travel: Replay a Thread Demo

- This means we load back in a checkpoint id (snapshot) from a point in our graph and start the graph from that point again. **This creates a fork.**
1. Run the graph with initial inputs using `invoke` or `stream`
 2. Use `get_state_history` to find a `checkpoint_id` you want to fork from (or set an interrupt at a point to stop at)
 3. Use the `invoke` or `stream` methods with an input of `None` and a configuration containing the appropriate `thread_id` and `checkpoint_id`
- To do this pass in:
 - `thread_id` is the ID of a thread.
 - `checkpoint_id` is an identifier that refers to a specific checkpoint within a thread.
 - `None` as the first argument of `invoke`

```
config = {"configurable": {"thread_id": "1", "checkpoint_id": "0c62ca34-ac19-445d-bbb0-5b4984975b2a"}}
graph.invoke(None, config=config) # or use graph.stream
```

▼ Time-Travel: Replay & Modify a Thread Demo

- So we can roll back to a point of the graph and get the state and change it then run the graph from this point with the modified state. **This creates a fork.**
1. Run the graph with initial inputs using `invoke` or `stream`
 2. Use `get_state_history` to find a `checkpoint_id` you want to fork from (or set an interrupt at a point to stop at)
 3. Use the `update_state` method to modify the graph's state at the checkpoint and resume execution from alternative state. We pass in:
 - `config` :
 - Containing the `thread_id` to update
 - When only the `thread_id` is passed, we update (or fork) the current state. Optionally, if we include `checkpoint_id` field, then we fork that selected checkpoint
 - `values`:
 - These are the values that will be used to update the state. Note that this update is treated exactly as any update from a node is treated. This means that these values will be passed to the **reducer** functions
 - This means that `update_state` does NOT automatically overwrite the channel values for every channel, but only for the channels without reducers. Let's walk through an example. (since default behaviour is overwrite)
 4. Use the `invoke` or `stream` methods with an input of `None` and a configuration containing the appropriate `thread_id` and `checkpoint_id`

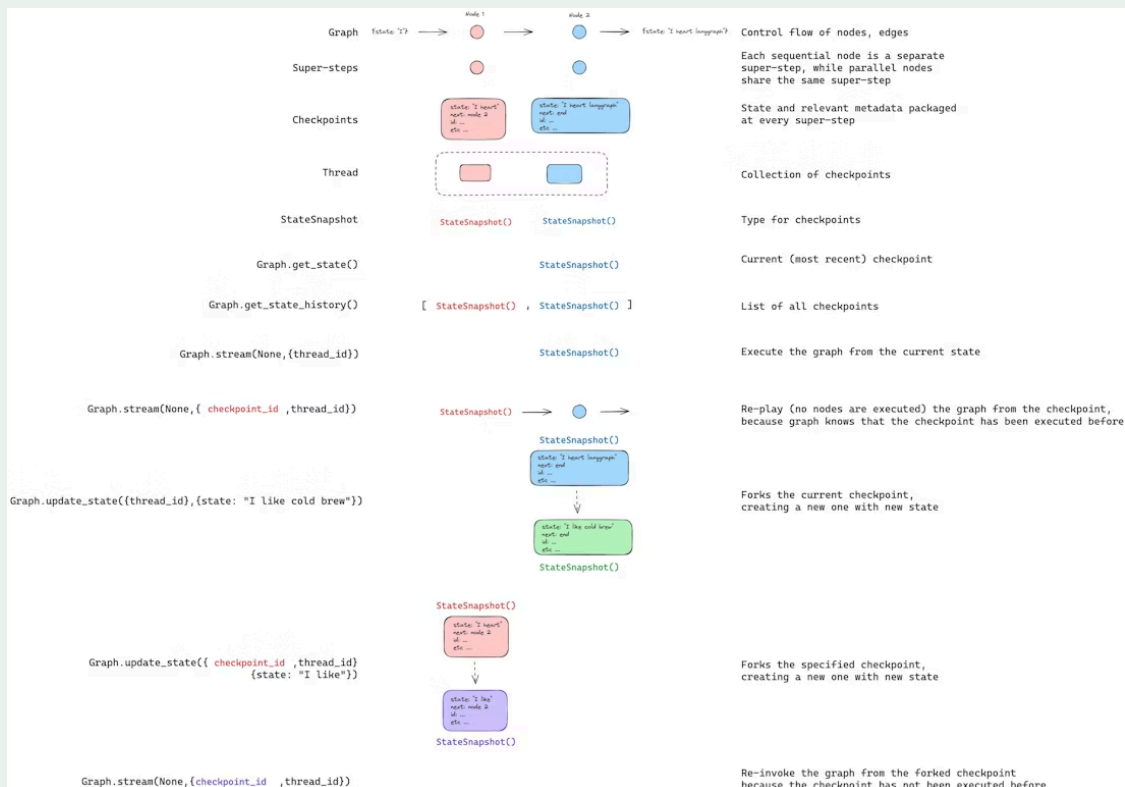
so if our state at the configured snapshot was
{"foo": 1, "bar": ["a"]}

if you update state as below
graph.update_state(config, {"foo": 2, "bar": ["b"]})

our next state will be
{"foo": 2, "bar": ["a", "b"]}

The foo key (channel) is completely changed (because there is no reducer specified for that channel, so update_state overwrites it).

However, there is a reducer specified for the bar key, and so it appends "b" to the state of bar.



Managing Short-Term Memory

▼ `trim_messages` Demo

- Here we truncate tokens above a set limit at the `call_model` node

```
from langchain_core.messages.utils import (
    trim_messages,
    count_tokens_approximately
)
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, MessagesState

model = init_chat_model("claude-sonnet-4-5-20250929")
summarization_model = model.bind(max_tokens=128)

def call_model(state: MessagesState):
    messages = trim_messages(
        state["messages"],
        strategy="last",
        token_counter=count_tokens_approximately,
        max_tokens=128,
        start_on="human",
        end_on=("human", "tool"),
    )
    response = model.invoke(messages)
    return {"messages": [response]}

checkpointer = InMemorySaver()
builder = StateGraph(MessagesState)
builder.add_node(call_model)
builder.add_edge(START, "call_model")
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "1"}}
graph.invoke({"messages": "hi, my name is bob"}, config)
```



```
graph.invoke({"messages": "write a short poem about cats"}, con
fig)
graph.invoke({"messages": "now do the same but for dogs"}, con
fig)
final_response = graph.invoke({"messages": "what's my name?"},
config)

final_response["messages"][-1].pretty_print()
```

▼ delete_messages Demo

- Here we use `.addsequence` which runs a function after a node for us
 1. When the graph reaches `call_model`, it runs it.
 2. As soon as that finishes, the output state is fed into `delete_messages`.
 3. The output of `delete_messages` becomes the updated state for the next step.
- This is a convenient way to manage context without introducing a new node just to do it

```
from langchain.messages import RemoveMessage

def delete_messages(state):
    messages = state["messages"]
    if len(messages) > 2:
        # remove the earliest two messages
        return {"messages": [RemoveMessage(id=m.id) for m in mes
sages[:2]]}

def call_model(state: MessagesState):
    response = model.invoke(state["messages"])
    return {"messages": response}
```

```

builder = StateGraph(MessagesState)
builder.add_sequence([call_model, delete_messages])
builder.add_edge(START, "call_model")

checkpointer = InMemorySaver()
app = builder.compile(checkpointer=checkpointer)

for event in app.stream(
    {"messages": [{"role": "user", "content": "hi! I'm bob"}]},
    config,
    stream_mode="values"
):
    print([(message.type, message.content) for message in event
["messages"]])

for event in app.stream(
    {"messages": [{"role": "user", "content": "what's my name?"}]},
    config,
    stream_mode="values"
):
    print([(message.type, message.content) for message in event
["messages"]])

[('human', "hi! I'm bob")]
[('human', "hi! I'm bob"), ('ai', 'Hi Bob! How are you doing today? I
s there anything I can help you with?')]
[('human', "hi! I'm bob"), ('ai', 'Hi Bob! How are you doing today? I
s there anything I can help you with?'), ('human', "what's my nam
e?")]
[('human', "hi! I'm bob"), ('ai', 'Hi Bob! How are you doing today? I
s there anything I can help you with?'), ('human', "what's my nam

```

```
e?"), ('ai', 'Your name is Bob.')]
[('human', "what's my name?"), ('ai', 'Your name is Bob.）]
```



When deleting messages, make sure that the resulting message history is valid. Check the limitations of the LLM provider you're using. For example:

- Some providers expect message history to start with a `user` message
- Most providers require `assistant` messages with tool calls to be followed by corresponding `tool` result messages.

▼ Summarisation Demo

```
from typing import Any, TypedDict

from langchain.chat_models import init_chat_model
from langchain.messages import AnyMessage
from langchain_core.messages.utils import count_tokens_approximately
from langgraph.graph import StateGraph, START, MessagesState
from langgraph.checkpoint.memory import InMemorySaver
from langmem.short_term import SummarizationNode, RunningSummary

model = init_chat_model("claude-sonnet-4-5-20250929")
summarization_model = model.bind(max_tokens=128)

class State(MessagesState):
    context: dict[str, RunningSummary]

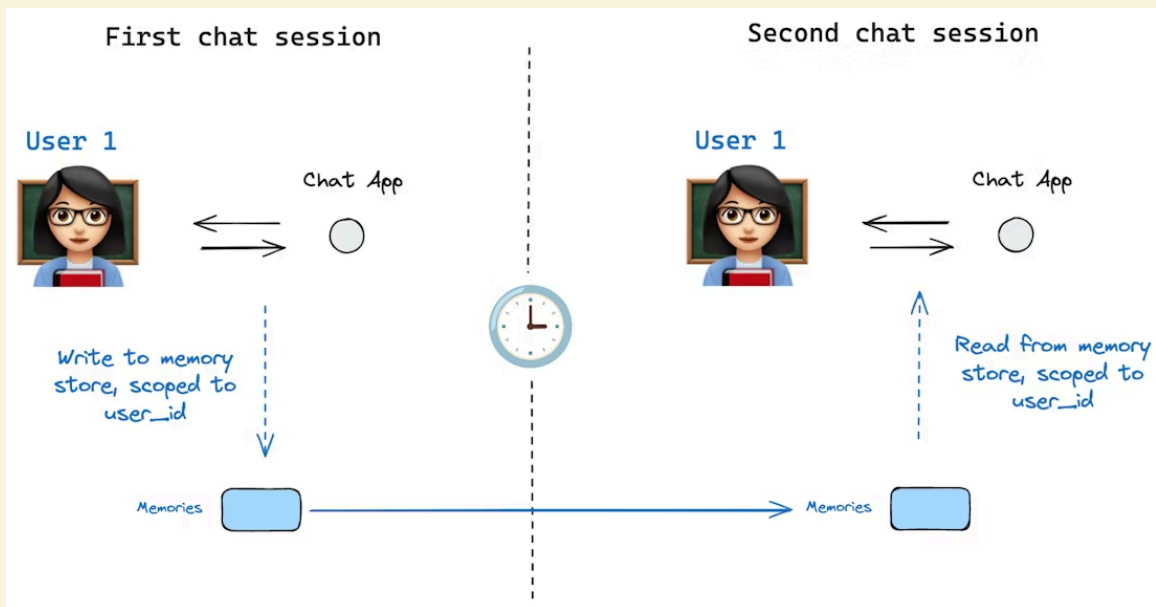
class LLMInputState(TypedDict):
    summarized_messages: list[AnyMessage]
    context: dict[str, RunningSummary]
```

```
summarization_node = SummarizationNode(  
    token_counter=count_tokens_approximately,  
    model=summarization_model,  
    max_tokens=256,  
    max_tokens_before_summary=256,  
    max_summary_tokens=128,  
)  
  
def call_model(state: LLMInputState):  
    response = model.invoke(state["summarized_messages"])  
    return {"messages": [response]}  
  
checkpointer = InMemorySaver()  
builder = StateGraph(State)  
builder.add_node(call_model)  
builder.add_node("summarize", summarization_node)  
builder.add_edge(START, "summarize")  
builder.add_edge("summarize", "call_model")  
graph = builder.compile(checkpointer=checkpointer)
```

Long-Term Memory

For this LangGraph uses stores...

- Stores are a in-built implemetation of long-term memory where LangGraph will just bring memories into state. We can perform retrieval on them.
- This lets us share information across threads
- The memory store is naepscaed as a tuple. For example:
 - (user_id)
 - or (user_id, "memories")
 - or (user_id, "memories", "growing up")



▼ InMemoryStore Demo

```
from langgraph.store.memory import InMemoryStore
store = InMemoryStore()

graph = graph.compile(checkpointer=checkpointer, store=in_memory_store)
```

```
user_id = "1"
namespace_for_memory = (user_id, "memories")
```

▼ PostgreSQLStore Demo

- We do not have to use an in-memory store, we can use a database memory store (MySQL, PostgreSQL, redis etc)

```
from langgraph.store.postgres import PostgresStore

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?
sslmode=disable"
with PostgresStore.from_conn_string(DB_URI) as store:
    builder = StateGraph(...)
    graph = builder.compile(store=store)
```

▼ Stores in Nodes Demo

```
# We will set a user id to use in our namespace for the store
config = {"configurable": {"thread_id": "1", "user_id": user_id}}

# First let's just say hi to the AI
for update in graph.stream(
    {"messages": [{"role": "user", "content": "hi"}]}, config, stream_mode="updates"
):
    print(update)

def update_memory(state: MessagesState, config: RunnableConfig, *,
store: BaseStore):

    # Get the user id from the config
    user_id = config["configurable"]["user_id"]
```

```

# Namespace the memory
namespace = (user_id, "memories")

# ... Analyze conversation and create a new memory

# Create a new memory ID
memory_id = str(uuid.uuid4())

# We create a new memory
store.put(namespace, memory_id, {"memory": memory})

```

- To access memories and use them in model calls in node functions:

```

def call_model(state: MessagesState, config: RunnableConfig, *, store: BaseStore):
    # Get the user id from the config
    user_id = config["configurable"]["user_id"]

    # Namespace the memory
    namespace = (user_id, "memories")

    # Search based on the most recent message
    memories = store.search(
        namespace,
        query=state["messages"][-1].content,
        limit=3
    )
    info = "\n".join([d.value["memory"] for d in memories])

    # ... Use memories in the model call

```

- Now if we create a new thread, we can still access the same memories so long as the `user_id` is the same

```
# Invoke the graph
config = {"configurable": {"thread_id": "2", "user_id": "1"}}

# Let's say hi again
for update in graph.stream(
    {"messages": [{"role": "user", "content": "hi, tell me about my memories"}]}, config, stream_mode="updates"
):
    print(update)
```

▼ **Add** Memory to Stores & **Read** From Stores Demo

- We use the `store.put` method to save memories to our namespace in the store.

```
memory_id = str(uuid.uuid4())
memory = {"food_preference": "I like pizza"}
in_memory_store.put(namespace_for_memory, memory_id, memory)
```

- We can read out memories in our namespace using the `store.search` method, which will return all memories for a given user as a list. The most recent memory is the last in the list.

```
memories = in_memory_store.search(namespace_for_memory)
memories[-1].dict()
{'value': {'food_preference': 'I like pizza'},
 'key': '07e0caf4-1631-47b7-b15f-65515d4c1843',
 'namespace': ['1', 'memories'],
 'created_at': '2024-10-02T17:22:31.590602+00:00',
 'updated_at': '2024-10-02T17:22:31.590605+00:00'}
```

▼ **Semantic** Retrieval Over Stores Demo


```

from langchain.embeddings import init_embeddings

store = InMemoryStore(
    index={
        "embed": init_embeddings("openai:text-embedding-3-small"), #
        "dims": 1536, # Embedding dimensions
        "fields": ["food_preference", "$"] # Fields to embed
    }
)

graph = graph.compile(checkpointer=checkpointer, store=in_memory_store)

```

- Now when searching, you can use natural language queries to find relevant memories:

```

# Find memories about food preferences
# (This can be done after putting memories into the store)
memories = store.search(
    namespace_for_memory,
    query="What does the user like to eat?",
    limit=3 # Return top 3 matches
)

```

- We can then choose if we want to `put` memories into vector db or just have them as retrievable but not searched

```

# Store with specific fields to embed
store.put(
    namespace_for_memory,
    str(uuid.uuid4()),
    {
        "food_preference": "I love Italian cuisine",
        "context": "Discussing dinner plans"
    }
)

```

```

    },
    index=["food_preference"] # Only embed "food_preferences" field
  )

# Store without embedding (still retrievable, but not searchable)
store.put(
  namespace_for_memory,
  str(uuid.uuid4()),
  {"system_info": "Last updated: 2024-01-01"},
  index=False
)

```



LangSmith Quirk

When we use the LangSmith, either locally (e.g., in Studio) or hosted with LangSmith, the base store is available to use by default and does not need to be specified during graph compilation. To enable semantic search, however, you do need to configure the indexing settings in your `langgraph.json` file.

▼ Example `langgraph.json`

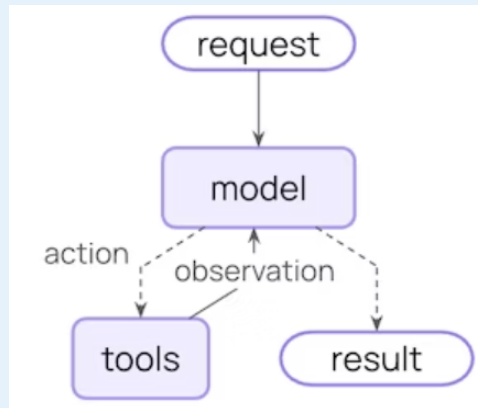
```

{
  ...
  "store": {
    "index": {
      "embed": "openai:text-embeddings-3-small",
      "dims": 1536,
      "fields": ["$"]
    }
  }
}

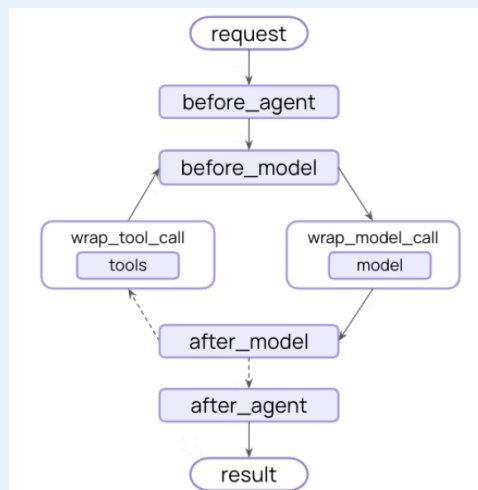
```

Middleware

<https://docs.langchain.com/oss/python/langchain/middleware/built-in>



Without middlewar



With middleware

- We can specify built in LangChain middleware which we pass into a `create_agent` call to add functionality to basic ReACT agents. For built-in middleware we do not need to specify where the middleware happens
- We can also build custom middleware and use it in our agent. For this we have different types of hooks:
 - **Node-style** (runs at each node) for sequential logic (logging, validation)

- `before_agent` - Before agent starts (once per invocation)
- `before_model` - Before each model call
- `after_model` - After each model response
- `after_agent` - After agent completes (once per invocation)
- **Wrap-style** (wraps each model call) for control flow (retry, fallback, caching)
 - `wrap_model_call` - Around each model call
 - `wrap_tool_call` - Around each tool call
- **Other**
 - `@dynamic_prompt` - Generates dynamic system prompts

Middleware	Description
<u>Summarization</u>	Automatically summarize conversation history when approaching token limits.
<u>Human-in-the-loop</u>	Pause execution for human approval of tool calls.
<u>Model call limit</u>	Limit the number of model calls to prevent excessive costs.
<u>Tool call limit</u>	Control tool execution by limiting call counts.
<u>Model fallback</u>	Automatically fallback to alternative models when primary fails.
<u>PII detection</u>	Detect and handle Personally Identifiable Information (PII).
<u>To-do list</u>	Equip agents with task planning and tracking capabilities.
<u>LLM tool selector</u>	Use an LLM to select relevant tools before calling main model.
<u>Tool retry</u>	Automatically retry failed tool calls with exponential backoff.
<u>Model retry</u>	Automatically retry failed model calls with exponential backoff.
<u>LLM tool emulator</u>	Emulate tool execution using an LLM for testing purposes.
<u>Context editing</u>	Manage conversation context by trimming or clearing tool uses.
<u>Shell tool</u>	Expose a persistent shell session to agents for command execution.
<u>File search</u>	Provide Glob and Grep search tools over filesystem files.

Vender-agnostic built in middleware is run

- `before_model` : will run before model calls, can update state or jump to other nodes.
- `after_model` : will run after model calls, can update state or jump to other nodes.
- `modify_model_request` : will run before model calls, allows user to modify (only for that model request) the tools, prompt, message list, model, model settings, output format, and tool choice.

LangChain's agent implementation is built on top of **LangGraph**; you still get durable execution, streaming, persistence and human-in-the-loop via LangGraph, but middleware gives you a neat, high-level place to plug cross-cutting concerns

so sounds like it brings some langgraph concept to langchain?

- lets you modify existing agents without changing everything by adding a middleware part

The core agent loop will still consist of a model node and a tool node. But Middleware can now specify:

- `before_model` : mutate state or **jump** to a different node (`model` , `tools` , `__end__`)
- `modify_model_request` : **adjust** the current request (model, system prompt, tool choice, response format) in a **stateless** way
- `after_model` : run **after** the model output but **before** tools execute; can update state or **jump** as well.

`before model` use for:

- Updating permanent state
- Summarisation

- Redirecting execution using `jump_to` with values: `"model"`, `"tools"`, or `"__end__"`
- Implementing complex decision logic
- State validation and preprocessing

`modify_model_request` use for:

- The `modify_model_request` hook operates immediately before model calls but specifically allows modification of tools, prompts, message lists, model parameters, model settings, output formats, and tool choice for that particular request. Unlike `before_model`, this function cannot modify permanent state or jump to different nodes.
- specify Anthropic's prompt caching

after)model use for:

- The `after_model` hook runs following model calls, offering opportunities for post-processing, result validation, or additional state modifications. Also allows human-on-the-loop This hook executes after model completion but before tool execution. Like `before_model`, it can use `jump_to` for flow control:

middleware lets us run logic before the model, tweak the actual model call and react to the model output before tools run, without rewriting the agent.

Middleware

provides powerful extensibility for customizing agent behavior at different stages of execution. You can use middleware to:

- Process state before the model is called (e.g., message trimming, context injection)
- Modify or validate the model's response (e.g., guardrails, content filtering)
- Handle tool execution errors with custom logic
- Implement dynamic model selection based on state or context

- Add custom logging, monitoring, or analytics

